

Find the best visual attempt

In this guide, you'll run a few variations of one task and keep the best result in a few lines. The sample task is an infographic prompt. The same pattern works for bug fixes, UI directions, prompts, queries, migrations, and other work where stronger alternatives are worth comparing.

Setup

Load the launch helpers.

```
In [1]: import pathlib
import sys

def _find_visual_artifact_example_root():
    cwd = pathlib.Path.cwd().resolve()
    candidates = []
    for base in (cwd, *cwd.parents):
        candidates.append(base)
        candidates.append(base / "examples" / "notebooks" / "visual_artifact")
    for candidate in candidates:
        if (candidate / "shepherd_usecases" / "visual_artifact" / "launch.py").exists():
            return candidate
    raise RuntimeError(
        "Cannot find examples/notebooks/visual_artifact. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    )

example_root = _find_visual_artifact_example_root()
if str(example_root) not in sys.path:
    sys.path.insert(0, str(example_root))
try:
    from shepherd_usecases.visual_artifact import launch
    from shepherd_usecases.visual_artifact import viz
except Exception as exc:
    raise RuntimeError(
        "Could not import the visual-artifact notebook helpers. "
        "Launch JupyterLab from the repository root with `make notebooks`."
    ) from exc

launch.bootstrap(example_root=example_root)
```

What this run does

Variant Studio is a best-of-N run: it turns one prompt into several isolated attempts, each in its own run, then hands the finished attempts to a reviewer that keeps the strongest.

The input

For this example, the run starts with one prompt and two short variant instructions.

```
In [2]: prompt = launch.default_prompt()
        variants = launch.variant_prompts()
        {"prompt": prompt, "variants": variants}
```

```
Out[2]: {'prompt': 'Please output an infographic tile explaining gradient descen
t.',
        'variants': {'contour-map': 'Use a contour-map visual with the update path
descending toward the minimum.',
        'uphill-path': 'Use an update-path visual that moves uphill away from the
minimum.'}}
```

Run Shepherd

First, open a workspace for the prompt.

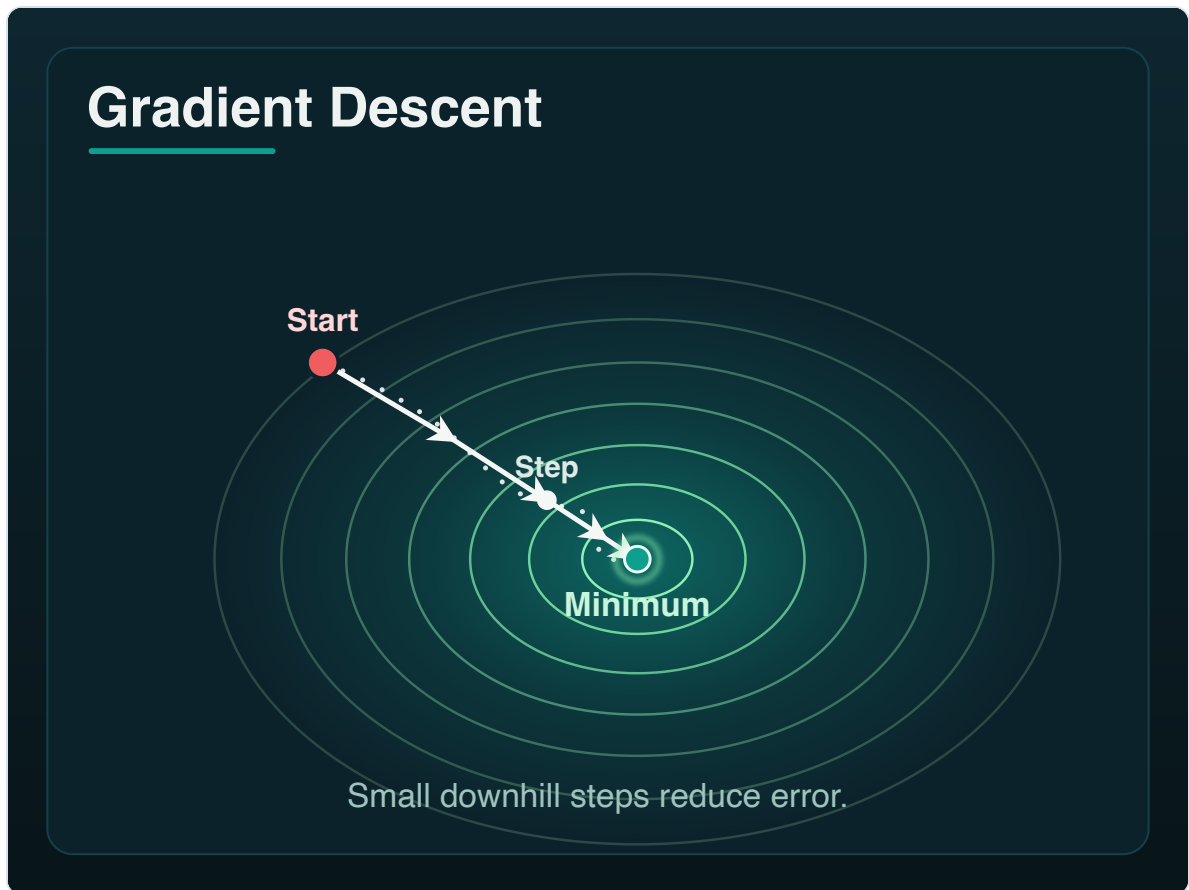
```
In [3]: workspace = launch.open_workspace("visual-variant-studio", prompt=prompt, me
```

Now run one attempt per variant from that workspace. Each run is isolated, so the two attempts write their artifacts without seeing each other.

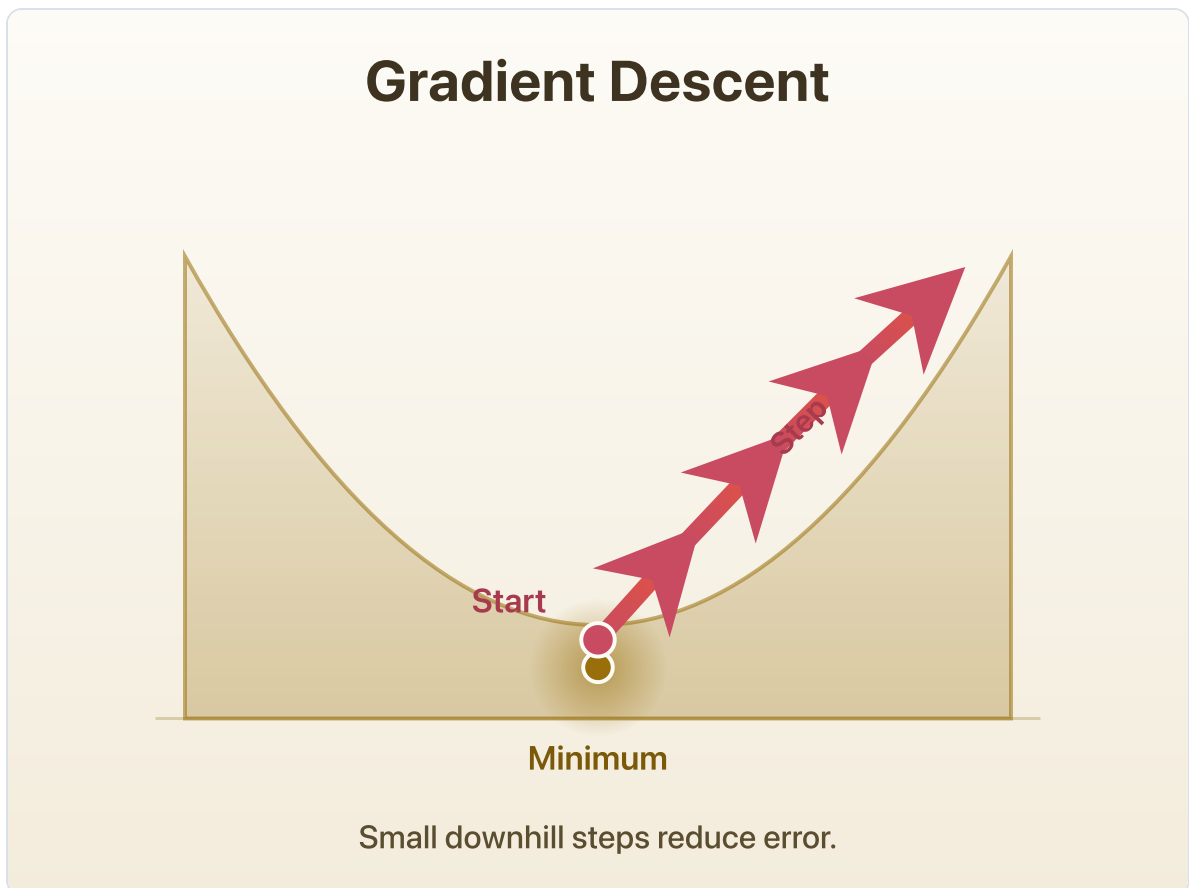
```
In [4]: attempts = {}
        for variant, instruction in variants.items():
            attempts[variant] = launch.run_static(
                workspace,
                name=variant,
                output_path=launch.ARTIFACT_PATH,
                output_text=launch.variant_html(prompt, variant),
                metadata={"variant": variant, "instruction": instruction},
            )

        viz.show(viz.run_artifacts(attempts))
```

contour-map



uphill-path



With both attempts finished, run the reviewer task. It receives the completed attempt runs as artifact references — `candidate_refs` — so it can read each one's artifact and pick the best.

```
In [5]: candidate_refs = {
        f"candidate_{name}": launch.artifact_ref(run, label=name)
        for name, run in attempts.items()
    }
    reviewer = launch.run_with_artifact_refs(
        workspace,
        name="review",
        refs=candidate_refs,
        output_path=launch.VERDICT_PATH,
        output_content=launch.review_content(prompt, attempts),
        after=list(attempts.values()),
    )
    selection = launch.selection_from_review(reviewer)
    viz.show(viz.review_summary(selection.candidates, selected=selection.selected))
```

Selected: contour-map

candidate	verdict	issues
contour-map	pass	
uphill-path	fail	descent path moves uphill away from the minimum

```
In [6]: for name, run in attempts.items():
        if name == selection.selected:
            run.output().select()
        else:
            run.output().discard()
    reviewer.output().release()
```

```
Out[6]: RetainedOutputSettlementResult(scope=ScopeInfo(name='run-43c9cd1e06ad', ref='refs/metagit/scopes/run-43c9cd1e06ad', instance_id='dc134b72939b', creation_oid='467e0c8a2a6943e01563a03c1c445aaaf214c6cc', world_id='world_0b9a3eb1990c'), parent=ScopeInfo(name='ground', ref='refs/metagit/ground', instance_id='ground-fd8c694b09c2', creation_oid='', world_id='world_8c81e4a6e505'), output_world_oid='99924a825b16e974907d1901ffdaf400267918eb', parent_world_before='f3508af87da7a45ac6b3edf35dd96eb7da760347', parent_world_after='f3508af87da7a45ac6b3edf35dd96eb7da760347', settlement=RetainedOutputSettlement(scope_name='run-43c9cd1e06ad', scope_ref='refs/metagit/scopes/run-43c9cd1e06ad', scope_instance_id='dc134b72939b', parent_ref='refs/metagit/ground', handoff_ref='refs/metagit/seals/b64u_cnVuLTQzYzljZDFlMDZhZA/b64u_ZGMxMzRiNzI5Mzli', output_world_oid='99924a825b16e974907d1901ffdaf400267918eb', binding='workspace', store_id='store_workspace', resource_id='fs:repo-main', candidate_id='primary', candidate_head='3e4cb908ab2cfb608a6fcbea9f85a1e5e94c63a6', action='released', operation_id='release_retained_40d2f767e1b27f9daa66f203654940ee', parent_world_before='f3508af87da7a45ac6b3edf35dd96eb7da760347', parent_world_after='f3508af87da7a45ac6b3edf35dd96eb7da760347', settlement_ref='refs/metagit/settlements/b64u_cnVuLTQzYzljZDFlMDZhZA/b64u_ZGMxMzRiNzI5Mzli/b64u_d29ya3NwYWNL/b64u_cHJpbWVyeQ', p030_authority_operation_id=None, p030_authority_settlement_operation_id=None, p030_authority_outcome=None))
```

Trace

How does Shepherd keep the attempts isolated and let one task read another's work?

The trace below captures every action an agent took while executing the tasks above, plus the relationships among those actions. Click the nodes to learn more about the events.

```
In [7]: notes = {run.run_ref: name for name, run in attempts.items()}
notes[reviewer.run_ref] = f"review: selected {selection.selected}"
viz.show(viz.trace(workspace.flow, {**attempts, "review": reviewer}, notes=r
```

Events			
kind	run/flow	label	payload
flow.opened	flow-acaeb779ca24		name='visual-variant-studio'
flow.fork.requested	run-f4db71c4c2dc	contour-map	name='contour-map', after=[]
run.lifecycle	run-f4db71c4c2dc	contour-map	task_id='shepherd_usecases.visua enforcement='advisory'
provider.invocation	run-f4db71c4c2dc	contour-map	execution_id='task-execution-9f4 provider_event_kind='provider.in evidence_role='provider_provenan 'sha256:1f5ce5df59a5b802457ccd79 'shepherd_usecases.visual_artifa
provider.invocation	run-f4db71c4c2dc	contour-map	execution_id='task-execution-9f4 provider_event_kind='provider.in evidence_role='provider_provenan 'index.html', 'content_digest': 'launched_confined': False}
run.output.published	run-f4db71c4c2dc	contour-map	output_name='workspace', output_ binding='workspace', output_worl
run.output.settled	run-f4db71c4c2dc	contour-map	output_name='workspace', output_ state='selected', settlement_ref='refs/metagit/set
flow.fork.requested	run-0f6221bf48f5	uphill-path	name='uphill-path', after=[]
run.lifecycle	run-0f6221bf48f5	uphill-path	task_id='shepherd_usecases.visua enforcement='advisory'
provider.invocation	run-0f6221bf48f5	uphill-path	execution_id='task-execution-c5f provider_event_kind='provider.in evidence_role='provider_provenan 'sha256:3ae7e97942494ef4e41e9460 'shepherd_usecases.visual_artifa

Optional Live Claude Run

The default path above is deterministic. Set the flag below only on a machine with the local Claude CLI, credentials, and native jail support.

```
In [8]: RUN_LIVE_CLAUDE = False
        CLAUDE_MODEL = None

        if RUN_LIVE_CLAUDE:
            launch.require_claude()
            live_workspace = launch.open_workspace("visual-variant-studio-live", pro
            try:
                live_attempts = {}
                for variant, instruction in variants.items():
                    live_attempts[variant] = launch.run_claude_artifact(
                        live_workspace,
```

```

        name=f"live-{variant}",
        prompt=prompt,
        variant=variant,
        instruction=instruction,
        model=CLAUDE_MODEL,
        metadata={"variant": variant, "mode": "live-claude"},
    )
    live_refs = {
        f"candidate_{name}": launch.artifact_ref(run, label=name)
        for name, run in live_attempts.items()
    }
    live_reviewer = launch.run_claude_review(
        live_workspace,
        name="live-review",
        prompt=prompt,
        refs=live_refs,
        after=list(live_attempts.values()),
        model=CLAUDE_MODEL,
        metadata={"mode": "live-claude"},
    )
    live_selection = launch.selection_from_review(live_reviewer)
    viz.show(viz.review_summary(live_selection.candidates, selected=live_selection.selected))
    for name, run in live_attempts.items():
        if name == live_selection.selected:
            run.output().select()
        else:
            run.output().discard()
    live_reviewer.output().release()
    viz.show(viz.trace(live_workspace.flow, {**live_attempts, "review": live_selection}))
finally:
    live_workspace.close()
else:
    launch.claude_preflight()

```

In [9]: `workspace.close()`

Want to understand how it works?

If you want to understand how this works in greater detail, open the [Variant Studio internals notebook](#).

Next steps

- Find the cheapest model that still does the job
- Recover a pipeline when one step drifts